# PROPOSAL PENELITIAN

# RANCANG BANGUN SISTEM SPEECH TO TEXT BERBASIS WEB MENGGUNAKAN MODEL OPENAI WHISPER

UIN IMAM BONJOL
PADANG

**Oleh:**

**MUHAMMAD IFFAN RAMADHAN**

**NIM. 2117020036**

**PROGRAM STUDI SISTEM INFORMASI**
**FAKULTAS SAINS DAN TEKNOLOGI**
**UNIVERSITAS ISLAM NEGERI IMAM BONJOL PADANG**
**2026**

# RANCANG BANGUN SISTEM SPEECH TO TEXT BERBASIS WEB MENGGUNAKAN MODEL OPENAI WHISPER

**PROPOSAL PENELITIAN**



UIN IMAM BONJOL
PADANG


**Oleh:**

**MUHAMMAD IFFAN RAMADHAN**

**NIM. 2117020036**


**PROGRAM STUDI SISTEM INFORMASI**
**FAKULTAS SAINS DAN TEKNOLOGI**
**UNIVERSITAS ISLAM NEGERI IMAM BONJOL PADANG**
**PADANG**
**2026**

## ABSTRAK

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.



**Kata Kunci:** Transkripsi, Whisper, SaaS, Benchmarking.

# HALAMAN PERSETUJUAN

Proposal penelitian dengan judul **"Rancang Bangun Sistem Speech To Text Berbasis Web Menggunakan Model OpenAI Whisper"** yang disusun oleh MUHAMMAD IFFAN RAMADHAN NIM 2117020036, telah diperiksa dan disetujui untuk diajukan dalam seminar proposal pada Program Studi Sistem Informasi Fakultas Sains dan Teknologi Universitas Islam Negeri Imam Bonjol Padang.

Padang, ..... 2026

Menyetujui:

| Pembimbing I | Pembimbing II |
|---|---|
| **Ahmad Fauzi, M.T.I.** | **Ozzy Secio Riza, M.Kom.** |
| NIP. 198805092018011001 | NIP. 199310122019032015 |

## DAFTAR ISI


**ABSTRAK** ..... **iii**

**HALAMAN PERSETUJUAN** ..... **iv**

**DAFTAR TABEL** ..... **vi**

**DAFTAR GAMBAR** ..... **vii**

**BAB I PENDAHULUAN** ..... **1**

A Latar Belakang ..... 1

B Rumusan Masalah ..... 5

C Batasan Masalah ..... 6

D Tujuan Penelitian ..... 6

E Manfaat Penelitian ..... 7

**BAB II TINJAUAN PUSTAKA** ..... **9**

A Kerangka Teori ..... 9

B Literatur Review ..... 22

**BAB III METODE PENELITIAN** ..... **28**

A Jenis Penelitian ..... 28

B Setting Penelitian ..... 28

C Prosedur Penelitian ..... 29

D Data dan Sumber Data ..... 31

E Alat dan Instrumen Penelitian ..... 31

F Teknik Analisis Data ..... 32

G Jadwal Penelitian ..... 34

**BAB IV HASIL DAN PEMBAHASAN** ..... **36**

A Hasil Penelitian ..... 36

B Pembahasan ..... 36

**BAB V PENUTUP** ..... **37**

A Kesimpulan ..... 37

B Saran ..... 37

**DAFTAR PUSTAKA** ..... **38**

## DAFTAR TABEL

2.1 Simbol, Nama, dan Fungsi Notasi UML..... 18
2.2 Ringkasan Literatur Review ..... 22
3.1 Statistik Dataset Penelitian..... 31
3.2 Jadwal Penelitian (*Gantt Chart*) ..... 35

## DAFTAR GAMBAR

3.1 Alur Penelitian (*Flowchart*)..... 30

# BAB I
# PENDAHULUAN

## A. Latar Belakang

Pendokumentasian informasi dari rekaman audio, seperti rekaman siniar, wawancara penelitian, dan persidangan akademik, membutuhkan proses transkripsi teks sebagai langkah awal pengolahan data. Keberadaan teks transkripsi mempermudah pengindeksan, pencarian kata kunci, serta pengarsipan dokumen digital. Kendati demikian, proses transkripsi yang dilakukan secara manual membutuhkan alokasi waktu dan tenaga yang besar. Rata-rata seorang transkriptor profesional memerlukan waktu sekitar lima jam hanya untuk menyelesaikan satu jam rekaman audio dengan transkripsi verbatim yang akurat Christodoulidou , dkk. (n.d.). Akibat tingginya beban kerja tersebut, banyak arsip audio pada berbagai lembaga pendidikan dan penelitian belum terdokumentasikan ke dalam bentuk teks tertulis Imai, Chowdhury, and Stent (n.d.).

Beban tersebut tidak hanya muncul pada tahap pengetikan, tetapi juga pada proses mendengarkan ulang, menentukan batas segmen, menyesuaikan tanda baca, dan memeriksa istilah yang tidak terdengar jelas. Pada rekaman percakapan, pekerjaan transkripsi menjadi lebih kompleks karena penutur dapat berbicara dengan kecepatan berbeda, menggunakan ungkapan tidak baku, atau menghasilkan ucapan yang saling bertumpuk. Dalam kondisi demikian, transkriptor harus berulang kali berpindah antara pemutar audio dan dokumen teks. Proses tersebut meningkatkan waktu penyelesaian dan membuka peluang terjadinya ketidakkonsistenan penulisan antarkalimat.

Pemanfaatan keluaran ASR sebagai draf dapat mengurangi sebagian pekerjaan tersebut, tetapi manfaatnya bergantung pada kualitas hasil pengenalan. Gaur dkk. menunjukkan bahwa keluaran ASR yang terlalu banyak kesalahan tidak selalu mempercepat transkripsi. Pada tingkat kesalahan tertentu, waktu yang dibutuhkan untuk memperbaiki hasil mesin dapat lebih panjang daripada menulis transkripsi dari awal. Temuan ini menunjukkan bahwa sistem transkripsi tidak

cukup hanya menghasilkan teks, tetapi juga perlu menyediakan informasi yang membantu pengguna menilai, menelusuri, dan memperbaiki kesalahan secara cepat Gaur, Lasecki, Metze, and Bigham (n.d.).

Perkembangan model *Automatic Speech Recognition* (ASR) modern berbasis arsitektur *Transformer*, seperti model OpenAI *Whisper*, memberikan peluang besar untuk mengotomatisasi konversi suara ke teks. Model ini dilatih menggunakan 680.000 jam data audio multibahasa sehingga memiliki ketahanan yang baik terhadap variasi aksen dan derau lingkungan Radford , dkk. (n.d.). Namun, penerapan model ASR pada rekaman percakapan alami berbahasa Indonesia menghadapi tantangan tersendiri. Karakteristik percakapan spontan yang memuat variasi dialek daerah, alih kode (*code-switching*), bahasa percakapan non-formal, serta tumpang tindih suara antar-pembicara kerap menurunkan tingkat akurasi transkripsi otomatis. Oleh karena itu, hasil transkripsi ASR tidak dapat langsung diposisikan sebagai dokumen final, melainkan memerlukan proses verifikasi dan penyuntingan manual oleh manusia. Pendekatan *human-in-the-loop* (HITL) terbukti mampu menjaga integritas data teks sekaligus meningkatkan efisiensi waktu kerja secara signifikan, dengan syarat didukung oleh sistem antarmuka penyuntingan yang ergonomis dan tersinkronisasi Olev and Alumäe (n.d.).

Kinerja ASR juga dipengaruhi oleh karakteristik bahan uji. Sistem yang bekerja baik pada ucapan terarah belum tentu memberikan hasil yang sama pada siniar atau percakapan spontan. Imai dkk. mengevaluasi beberapa sistem ASR sumber terbuka pada kondisi audio yang beragam dan menunjukkan bahwa kebisingan, karakteristik penutur, serta tumpang tindih suara dapat memengaruhi nilai kesalahan transkripsi. Pada konteks siniar, Teleki dkk. juga menunjukkan bahwa pilihan sistem ASR perlu disesuaikan dengan jenis disfluensi yang terdapat dalam rekaman. Perbedaan antara pengulangan kata, interjeksi, dan bagian ucapan yang telah diedit dapat menghasilkan pola kesalahan yang berbeda. Karena itu, evaluasi sistem perlu menggunakan data percakapan yang mendekati kondisi penggunaan sebenarnya, bukan hanya data dengan artikulasi yang bersih Imai , dkk. (n.d.); Teleki, Dong, Kim, and Caverlee (n.d.).

Masalah lain terletak pada pengolahan audio berdurasi panjang. Model ASR umumnya memproses audio dalam segmen tertentu, sedangkan siniar yang digunakan dalam penelitian ini disimpan sebagai berkas utuh per episode. Pemotongan yang tidak tepat dapat memisahkan satu rangkaian ujaran, menghilangkan konteks di sekitar batas segmen, atau menghasilkan tanda waktu yang bergeser. WhisperX mengatasi sebagian persoalan tersebut melalui deteksi aktivitas suara, pemrosesan berkelompok, dan *forced alignment* pada tingkat kata. Pendekatan ini memperlihatkan bahwa akurasi transkripsi dan ketepatan waktu keluaran perlu dipertimbangkan secara bersamaan ketika sistem digunakan untuk menyunting rekaman panjang Bain, Huh, Han, and Zisserman (n.d.).

Dalam penggunaan berbahasa Indonesia, pemilihan model juga perlu mempertimbangkan keterbatasan sumber daya komputasi dan karakteristik bahasa. Model yang lebih besar cenderung membutuhkan memori dan waktu inferensi yang lebih tinggi, sedangkan model yang lebih kecil dapat menghasilkan kesalahan pada kata tidak baku, istilah khusus, atau bagian audio yang bising. Fathony dkk. menunjukkan bahwa optimasi dan penggunaan lebih dari satu mesin pemrosesan dapat dipertimbangkan untuk memperoleh keseimbangan antara akurasi dan efisiensi pada transkripsi bahasa Indonesia. Pertimbangan tersebut relevan bagi sistem yang ditujukan untuk digunakan secara mandiri, karena kapasitas perangkat keras dan biaya komputasi harus diperhitungkan sejak tahap perancangan Fathony, Mareta, Adiana, Wardhani, and Halim (n.d.).

Tantangan berikutnya berkaitan dengan aspek infrastruktur dan tata kelola data. Pemanfaatan layanan ASR komputasi awan (*cloud*) komersial membutuhkan biaya langganan yang tinggi berdasarkan durasi audio, serta menimbulkan kekhawatiran terkait kerahasiaan dan privasi data penelitian institusi Ferraro, Galli, La Gatta, and Postiglione (n.d.). Pengiriman berkas audio internal ke peladen (*server*) pihak ketiga di luar negeri berisiko melanggar ketentuan perlindungan data institusi lokal. Di sisi lain, pemanfaatan perangkat keras GPU mandiri (*self-hosted*) pada lingkungan lokal sering kali terkendala oleh manajemen beban komputasi model AI yang berat. Model inferensi ASR berbasis *Transformer* membutuhkan

alokasi memori VRAM yang besar dan waktu eksekusi yang intensif. Tanpa pemisahan arsitektur yang terstruktur, proses inferensi audio berdurasi panjang dapat menyebabkan kegagalan memori (*Out-of-Memory*) serta membekukan antarmuka aplikasi web Fathony , dkk. (n.d.).

Perbandingan antara layanan komersial dan perangkat lunak sumber terbuka menunjukkan adanya pertukaran antara kualitas, biaya, kendali data, dan kemudahan pengelolaan. Layanan komersial biasanya menyediakan infrastruktur siap pakai, tetapi pengguna bergantung pada kebijakan penyedia, koneksi jaringan, serta mekanisme penagihan berdasarkan pemakaian. Sistem sumber terbuka memberi ruang untuk menjalankan model di lingkungan sendiri dan mengatur alur pemrosesan sesuai kebutuhan, tetapi membutuhkan pengelolaan perangkat keras, penyimpanan, antrean pekerjaan, dan pemantauan proses. Ferraro dkk. menekankan bahwa kualitas layanan ASR dapat berubah sesuai variasi masukan, sehingga penilaian sistem perlu mencakup kondisi audio yang nyata dan bukan hanya perbandingan angka akurasi pada satu jenis data Ferraro , dkk. (n.d.).

Berdasarkan pertimbangan tersebut, sistem yang dibangun perlu memisahkan pekerjaan yang bersifat interaktif dari pekerjaan inferensi yang membutuhkan komputasi tinggi. Pengunggahan berkas, penampilan status, dan penyuntingan teks harus tetap responsif meskipun unit pemrosesan sedang menangani audio berdurasi panjang. Pemisahan ini juga memungkinkan pengujian beberapa varian model secara lebih teratur karena setiap pekerjaan dapat dicatat berdasarkan model, durasi audio, waktu inferensi, penggunaan VRAM, dan hasil transkripsinya. Dengan pencatatan tersebut, keputusan pemilihan model tidak hanya didasarkan pada ukuran parameter, tetapi juga pada kesesuaian antara akurasi, kecepatan, dan kebutuhan sumber daya.

Berdasarkan permasalahan tersebut, penelitian ini merancang dan membangun sebuah platform transkripsi audio mandiri bernama Skriptor. Sistem ini menerapkan arsitektur terdistribusi yang memisahkan antarmuka pengguna berbasis *Next.js* dengan unit pemrosesan inferensi berbasis *Python Worker*. Komunikasi asinkron dijembatani oleh sistem antrean tugas *Redis BullMQ*, sehingga proses inferensi model OpenAI *Whisper* dapat berjalan stabil di latar belakang tanpa

menghambat responsivitas antarmuka web. Melalui integrasi ini, pengguna dapat memantau progres transkripsi secara langsung serta melakukan koreksi teks secara interaktif melalui visualisasi gelombang audio. Dengan demikian, penelitian ini berfokus pada rancang bangun sistem *speech-to-text* berbasis *web* yang efisien dan terukur untuk alur kerja transkripsi audio percakapan bahasa Indonesia.

Selain menghasilkan teks, sistem ini dirancang untuk mendukung proses pemeriksaan oleh pengguna. Hasil transkripsi perlu disajikan bersama informasi waktu dan bagian audio yang terkait agar kesalahan dapat ditelusuri kembali ke sumbernya. Antarmuka yang demikian menempatkan model ASR sebagai alat bantu, sedangkan keputusan akhir mengenai ketepatan isi tetap berada pada pengguna. Pendekatan tersebut sesuai dengan karakteristik data percakapan yang sering memuat nama diri, istilah lokal, dan konteks yang tidak dapat ditentukan hanya dari sinyal suara.

Dengan mempertimbangkan kebutuhan tersebut, penelitian ini tidak hanya menguji kemampuan model dalam menghasilkan teks, tetapi juga mengukur efisiensi komputasi dan kegunaan antarmuka. Akurasi dinilai menggunakan WER dan CER, efisiensi dihitung melalui RTF serta penggunaan VRAM, sedangkan penerimaan antarmuka dievaluasi menggunakan SUS. Kombinasi pengukuran tersebut diharapkan memberikan gambaran yang lebih lengkap mengenai kelayakan sistem transkripsi mandiri untuk rekaman percakapan bahasa Indonesia.

## B. Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam penelitian ini dirumuskan sebagai berikut:

1. Bagaimana rancang bangun sistem *speech-to-text* berbasis *web* dengan arsitektur terdistribusi menggunakan *Next.js* dan *Python Worker*?
2. Bagaimana performa kecepatan (*inference time* dan *Real-Time Factor*), konsumsi VRAM, serta tingkat akurasi (*Word Error Rate* dan *Character Error Rate*) dari berbagai varian model OpenAI *Whisper* ketika diintegrasikan ke dalam arsitektur sistem terdistribusi tersebut?
3. Bagaimana tingkat kelayakan kegunaan (*usability*) antarmuka sistem

*speech-to-text* berbasis *web* yang dikembangkan berdasarkan evaluasi *System Usability Scale* (SUS)?

## C. Batasan Masalah

Penelitian ini dibatasi pada ruang lingkup sebagai berikut:

1. Model transkripsi yang diuji terbatas pada varian model OpenAI *Whisper* yang mencakup varian *Tiny, Base, Small, Medium, Large-v2, Large-v3,* dan *Large-v3-Turbo*.
2. Sistem dibangun menggunakan kerangka kerja *Next.js* 16 dengan bahasa *TypeScript* pada antarmuka web, serta unit *worker* berbasis *Python* yang terhubung melalui antrean terdistribusi berbasis *Redis BullMQ*.
3. Dataset pengujian menggunakan korpus percakapan bahasa Indonesia dari *Mozilla Data Collective* (MDC) yang terdiri dari 58 berkas audio utuh per episode siniar (*Homostoria* dan *Hari Minggoean*) dengan total durasi 23,5 jam, yang dievaluasi terhadap 11.540 segmen transkrip pada berkas acuan (*ground truth*) bertaraf TSV.
4. Infrastruktur inferensi pengujian performa dijalankan pada lingkungan komputasi berakselerasi GPU NVIDIA RTX A5000 (24GB VRAM) pada peladen *Runpod*.

## D. Tujuan Penelitian

Tujuan yang hendak dicapai dalam penelitian ini adalah:

1. Merancang dan membangun sistem *speech-to-text* berbasis *web* dengan arsitektur terdistribusi yang memisahkan lapisan aplikasi (*Next.js*) dan lapisan komputasi kecerdasan buatan (*Python Worker*).
2. Mengevaluasi performa kecepatan inferensi, efisiensi konsumsi memori VRAM, dan tingkat akurasi transkripsi (WER dan CER) dari berbagai varian model OpenAI *Whisper* pada data rekaman percakapan bahasa Indonesia.
3. Mengukur tingkat kegunaan (*usability*) antarmuka sistem *speech-to-text* yang dikembangkan menggunakan instrumen kuesioner terstandarisasi *System Usability Scale* (SUS) terhadap 30 orang responden.

## E. Manfaat Penelitian

Manfaat yang diharapkan dari hasil penelitian ini meliputi: Secara teoretis, penelitian ini memberikan data empiris dan analisis komparatif mengenai performa model ASR berbasis *Transformer* pada rekaman percakapan alami berbahasa Indonesia. Penelitian ini juga memperkaya kajian rekayasa perangkat lunak mengenai implementasi sistem AI terdistribusi dan penerapan prinsip *User-Centered Design* (UCD) pada antarmuka koreksi teks berbasis pendekatan *human-in-the-loop*.

Secara praktis, penelitian ini menghasilkan prototipe sistem transkripsi mandiri (*self-hosted*) bernama Skriptor yang dapat diterapkan oleh instansi akademis, lembaga riset, maupun profesional media. Sistem tersebut diharapkan membantu menurunkan biaya operasional transkripsi, menjaga kerahasiaan data audio lokal, dan mempercepat penyelesaian transkripsi melalui fasilitas penyuntingan yang terintegrasi.

– Active: 1788614897217@@mirx-server.lan@5432@skriptor

# BAB II
# TINJAUAN PUSTAKA

## A. Kerangka Teori

Bagian ini memaparkan teori-teori dasar yang menjadi landasan pengembangan dan analisis sistem transkripsi terdistribusi. Alur tinjauan dimulai dari konsep *Automatic Speech Recognition* (ASR) dan arsitektur model *Whisper* sebagai mesin transkripsi utama, dilanjutkan dengan karakteristik bahasa Indonesia yang menjadi konteks penelitian, kemudian *pipeline WhisperX* dan teknik optimasi inferensi yang digunakan, hingga metrik evaluasi, arsitektur sistem terdistribusi, aspek *usability*, dan pengujian perangkat lunak. Teori-teori ini menjadi dasar untuk merancang dan mengevaluasi sistem pada Bab 3.

### 1. Automatic Speech Recognition (ASR)

*Automatic Speech Recognition* (ASR) adalah teknologi yang memungkinkan komputer untuk mengubah sinyal suara menjadi teks secara otomatis. Proses ini melibatkan konversi gelombang akustik mentah menjadi representasi digital yang kemudian dipetakan ke dalam satuan bahasa (fonem, suku kata, atau kata). Secara tradisional, sistem ASR mengandalkan komponen terpisah seperti model akustik, model bahasa, dan leksikon pelafalan. Namun, perkembangan terkini telah bergeser ke arah model *End-to-End* (E2E) yang menyederhanakan alur kerja tersebut menjadi satu jaringan saraf tunggal yang melakukan pemetaan langsung dari audio ke teks Prabhavalkar, Hori, Sainath, Schlüter, and Watanabe (n.d.).

### 2. Arsitektur Model Whisper (OpenAI)

Model *Whisper* yang dikembangkan oleh OpenAI merupakan arsitektur jaringan saraf *End-to-End* (E2E) berbasis *Transformer Encoder-Decoder* yang dilatih menggunakan pendekatan *weakly supervised learning* pada skala data sebesar 680.000 jam audio multibahasa Radford , dkk. (n.d.). Arsitektur ini terdiri dari tiga komponen utama: pra-pemrosesan audio, *encoder*, dan *decoder* autoregresif.

Pada tahap pra-pemrosesan audio (*Audio Pre-processing*), sinyal audio mentah (*raw audio*) diubah menjadi berfrekuensi tunggal (*monophonic*) dengan laju sampel (*sampling rate*) 16.000 Hz. Sinyal tersebut kemudian dipecah menjadi potongan (*chunks*) berdurasi 30 detik. Ekstraksi fitur dilakukan menggunakan transformasi Fourier jangka pendek (*Short-Time Fourier Transform* atau STFT) untuk menghasilkan spektrogram log-Mel berukuran 80 dimensi, atau 128 dimensi pada varian *Whisper v3*.

*Encoder* Transformer memetakan spektrogram masukan menjadi representasi fitur akustik. Dimensi waktu potongan audio terlebih dahulu direduksi melalui blok konvolusi dan dilengkapi *positional embeddings*. Representasi tersebut kemudian diproses oleh tumpukan blok *Transformer Encoder* yang terdiri atas mekanisme *Multi-Head Self-Attention* (MHA) dan jaringan *Feed-Forward* (FFN) dengan normalisasi lapisan *Pre-LN*.

*Decoder* Transformer berfungsi sebagai model bahasa kondisional (*conditional language model*) yang memprediksi urutan token secara autoregresif berdasarkan representasi akustik dari *encoder*. Mekanisme *masked Multi-Head Self-Attention* mencegah model melihat token berikutnya, sedangkan *Multi-Head Cross-Attention* menghubungkan *decoder* dengan representasi akustik *encoder*.

Selain itu, model *Whisper* mengintegrasikan beberapa tugas pemrosesan suara melalui token kontrol khusus (*special tokens*) pada awal urutan *decoder*. Token tersebut mencakup penanda awal transkripsi, bahasa sasaran seperti $\langle|\text{id}|\rangle$, jenis tugas berupa *transcribe* atau *translate*, serta token kontrol tanda waktu.

### 3. Karakteristik Bahasa Indonesia untuk ASR

Bahasa Indonesia memiliki tantangan tersendiri dalam ASR karena variasi dialek dan statusnya yang sering dikategorikan sebagai *low-resource language* dalam beberapa korpus besar. Meskipun Whisper memiliki performa multibahasa yang baik, akurasinya pada bahasa Indonesia sering kali lebih rendah dibandingkan bahasa Inggris, sehingga memerlukan evaluasi spesifik menggunakan *dataset* lokal Fathony , dkk. (n.d.). Selain itu, percakapan alami berbahasa Indonesia, seperti pada

siniar, kerap mengandung bahasa gaul, ambiguitas linguistik, dan gangguan latar belakang yang menambah tingkat kesulitan transkripsi. Karakteristik ini menjadi dasar pemilihan korpus siniar berbahasa Indonesia sebagai objek pengujian dalam penelitian ini.

### 4. Inovasi *Pipeline* WhisperX

Meskipun *Whisper* memiliki kemampuan generalisasi yang sangat baik, model ini menghadapi keterbatasan serius ketika diterapkan pada audio berdurasi panjang, seperti masalah pergeseran tanda waktu (*timestamp drift*), halusinasi teks pada area hening (*hallucination loops*), dan latensi komputasi yang tinggi akibat sifat pemrosesan sekuensial autoregresif. Untuk mengatasi masalah tersebut, Bain , dkk. (n.d.) memperkenalkan *WhisperX*, sebuah kerangka kerja terpadu berkinerja tinggi yang menggabungkan deteksi aktivitas suara, inferensi berkelompok, dan penyelarasan fonem paksa.

Sebelum audio dikirim ke model ASR, *WhisperX* menerapkan *Voice Activity Detection* (VAD) dengan strategi *Cut & Merge*. VAD memisahkan segmen bicara aktif dari bagian hening, kemudian menggabungkan segmen aktif secara dinamis hingga panjang maksimal 30 detik. Dengan demikian, pemotongan berlangsung pada fase hening alami dan kecenderungan *Whisper* menghasilkan pengulangan frasa (*repetition loops*) dapat dikurangi.

Berbeda dari pemrosesan berurutan (*sequential decoding*) pada *Whisper* asli, *batched transcription* melalui *Faster-Whisper* memungkinkan paralelisasi komputasi pada GPU. Karena pembagian segmen berbasis VAD menghasilkan batas potongan yang bersih, setiap segmen dapat ditranskripsikan secara mandiri dan paralel dalam satu kelompok (*batch*). *WhisperX* mengintegrasikan *Faster-Whisper*, yaitu implementasi model *Whisper* dalam *C++* yang menggunakan pustaka optimasi *CTranslate2*. Kombinasi *batching* dan optimasi tersebut mempercepat waktu pemrosesan pada GPU NVIDIA modern.

Penanda waktu bawaan *Whisper* diprediksi pada tingkat frasa dengan presisi kasar dan rentan terhadap pergeseran temporal (*drifting*). Untuk memperoleh

penanda waktu tingkat kata (*word-level timestamp*) yang akurat, *WhisperX* menerapkan *forced alignment* menggunakan model berbasis *Connectionist Temporal Classification* (CTC), seperti *Wav2Vec 2.0* atau *WavLM*. Hasil pemetaan ini memberikan koordinat batas waktu awal dan akhir ($t_{start}, t_{end}$) setiap kata.

Tahap terakhir dalam *pipeline WhisperX* adalah *speaker diarization*, yaitu klasifikasi identitas suara untuk menentukan siapa yang berbicara pada setiap kata (*who spoke when*). Pustaka *PyAnnote Audio* mengekstraksi *speaker embeddings*, misalnya menggunakan *ECAPA-TDNN*, kemudian mengelompokkan pembicara dan memetakan identitasnya ke setiap kata berdasarkan koordinat waktu dari tahap *forced alignment*.

### 5. Optimasi Inferensi: Faster-Whisper dan Kuantisasi

Untuk menjalankan model besar pada lingkungan produksi yang efisien, diperlukan teknik optimasi inferensi yang mampu memangkas waktu pemrosesan dan konsumsi memori tanpa mendegradasi akurasi secara signifikan. Pustaka *CTranslate2* merupakan mesin inferensi berkinerja tinggi untuk model berarsitektur *Transformer*. Pustaka ini menerapkan peleburan operator (*operator fusion*), alokasi memori dinamis, serta paralelisasi CPU dan GPU untuk mengurangi *overhead* eksekusi *PyTorch*.

Kuantisasi model FP16 dan INT8 mengurangi presisi numerik bobot (*weights*) jaringan saraf. Pada inferensi FP32, setiap bobot membutuhkan 32 bit (4 bita), sedangkan INT8 merepresentasikannya dengan 8 bit (1 bita) melalui rumus:

$$W_q = \text{round}\left(\frac{W}{S} + Z\right) \tag{2.1}$$

di mana $W$ melambangkan bobot berpresisi pecahan kontinu, $S$ adalah faktor skala (*scale factor*), dan $Z$ merupakan titik nol (*zero-point*). Kuantisasi ini menghemat kapasitas memori VRAM GPU hingga 50–75% serta mempercepat perpindahan data pada memori Andreyev (n.d.). Kecepatan inferensi diukur menggunakan *Real-Time Factor* (RTF), yaitu perbandingan waktu komputasi model ($\Delta t_{\text{inferensi}}$) terhadap durasi

berkas audio ($\Delta t_{\text{audio}}$):

$$\text{RTF} = \frac{\Delta t_{\text{inferensi}}}{\Delta t_{\text{audio}}} \tag{2.2}$$

Nilai RTF $< 1{,}0$ menandakan proses inferensi berjalan lebih cepat dibandingkan durasi pemutaran audio di dunia nyata (*faster than real-time*). Kebalikan dari RTF dinyatakan sebagai faktor percepatan komputasi (*Speed Multiplier*):

$$\text{Speed} = \frac{1}{\text{RTF}} = \frac{\Delta t_{\text{audio}}}{\Delta t_{\text{inferensi}}} \tag{2.3}$$

### 6. Metrik Evaluasi Akurasi: WER dan CER

Akurasi sistem ASR dievaluasi secara kuantitatif berdasarkan jarak suntingan Levenshtein (*Levenshtein Distance*) yang mengukur perbedaan jumlah operasi minimum antara urutan token keluaran hipotesis model dengan urutan token berkas acuan manual (*ground truth*). *Word Error Rate* (WER) merupakan metrik utama untuk mengukur tingkat kesalahan pengenalan wicara pada tingkat kata. Nilai WER dihitung melalui persamaan:

$$\text{WER} = \frac{S+D+I}{N} = \frac{S+D+I}{S+D+C} \tag{2.4}$$

di mana:

- $S$ (*Substitutions*) adalah jumlah kata dalam referensi yang digantikan oleh kata lain pada keluaran model.
- $D$ (*Deletions*) adalah jumlah kata dalam referensi yang hilang atau tidak dikenali oleh model.
- $I$ (*Insertions*) adalah jumlah kata tambahan pada keluaran model yang sebenarnya tidak ada pada referensi.
- $C$ (*Correct*) adalah jumlah kata yang berhasil dikenali secara tepat dan identik.
- $N$ adalah total jumlah seluruh kata yang ada pada teks referensi acuan ($N = S+D+C$).

*Character Error Rate* (CER) merupakan metrik evaluasi kesalahan pada tingkat karakter yang dihitung dengan prinsip yang sama seperti WER:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c} \tag{2.5}$$

di mana $S_c$, $D_c$, $I_c$, dan $N_c$ dihitung pada satuan karakter individual. CER sangat esensial dalam analisis bahasa Indonesia untuk mendeteksi ketepatan imbuhan (afiksasi) dan partikel yang mungkin hanya menimbulkan sedikit variasi karakter namun mengubah makna kata secara keseluruhan Ferraro , dkk. (n.d.).

### 7. Arsitektur Sistem Terdistribusi

Sistem berskala besar yang menangani beban inferensi AI intensif memerlukan pemisahan tanggung jawab (*separation of concerns*) melalui arsitektur terdistribusi: Arsitektur *Message Queue Pattern* menempatkan peladen web sebagai produsen (*producer*) yang memasukkan pekerjaan ke dalam antrean terisolasi. Pustaka *BullMQ* berbasis *Redis* mengelola siklus hidup pekerjaan, mekanisme pengulangan saat gagal (*retry mechanism* dengan *exponential backoff*), serta penguncian tugas (*distributed locks*) untuk mencegah tabrakan eksekusi antar-pekerja.

Pada *Worker Pattern*, satu atau beberapa unit pekerja (*worker*) berjalan di lingkungan terpisah dan berfokus pada komputasi GPU. Pemisahan ini memungkinkan isolasi konkurensi, misalnya satu antrean untuk setiap GPU, sehingga penggunaan VRAM lebih stabil dan risiko galat kehabisan memori (*Out-of-Memory*) dapat dikurangi Macháček, Dabre, and Bojar (n.d.).

Progres inferensi disiarkan ke antarmuka klien melalui *Server-Sent Events* (SSE) yang dipicu oleh *Redis Pub/Sub*. Integritas data dari unit pemrosesan eksternal ke peladen web dijaga dengan verifikasi tanda tangan kriptografis menggunakan *Hash-based Message Authentication Code* (HMAC-SHA256).

## 8. Human-in-the-Loop (HITL), *Usability*, dan *System Usability Scale*

Konsep *Human-in-the-Loop* (HITL) mengakui bahwa sistem kecerdasan buatan, secanggih apa pun, tidak dapat sepenuhnya menggantikan penilaian manusia dalam tugas-tugas yang memerlukan pemahaman kontekstual dan ketelitian tinggi. Dalam konteks transkripsi, HITL menempatkan manusia sebagai validator akhir yang bertugas memeriksa dan mengoreksi hasil keluaran model ASR sebelum draf transkripsi dianggap final. Penelitian terdahulu menunjukkan bahwa alur kerja semi-otomatis yang menggabungkan draf transkripsi dari AI dengan antarmuka penyuntingan yang intuitif dapat meningkatkan produktivitas secara signifikan, terutama jika antarmuka tersebut dilengkapi dengan fitur visualisasi gelombang audio yang tersinkronisasi dengan teks Imai , dkk. (n.d.); Olev and Alumäe (n.d.). Namun, efektivitas pendekatan HITL sangat bergantung pada tingkat kegunaan (*usability*) antarmuka yang disediakan bagi pengguna.

Karena antarmuka koreksi menjadi titik temu utama antara manusia dan sistem, perancangannya perlu berpusat pada kebutuhan pengguna. Pendekatan *User-Centered Design* (UCD) menempatkan pengguna sebagai pusat proses desain, mulai dari pemahaman konteks penggunaan, perancangan solusi, hingga evaluasi Riebel (n.d.). Dalam penelitian ini, prinsip UCD diterapkan pada perancangan antarmuka koreksi Skriptor agar alur kerja penyuntingan hasil transkripsi dapat dilakukan secara intuitif dan efisien.

Menurut standar internasional ISO 9241-11, *usability* didefinisikan sebagai sejauh mana suatu produk dapat digunakan oleh pengguna tertentu untuk mencapai tujuan tertentu dengan efektivitas (*effectiveness*), efisiensi (*efficiency*), dan kepuasan (*satisfaction*) dalam konteks penggunaan tertentu. Untuk mengukur aspek *usability* secara kuantitatif pada sistem yang dikembangkan, diperlukan instrumen pengukuran yang terstandarisasi dan reliabel.

*System Usability Scale* (SUS) yang dikembangkan oleh Brooke (n.d.) merupakan kuesioner terstandarisasi yang terdiri dari 10 butir pernyataan dengan skala Likert 5 poin (1 = Sangat Tidak Setuju hingga 5 = Sangat Setuju). Kuesioner

ini dirancang untuk memberikan penilaian global terhadap kegunaan suatu sistem secara cepat dan murah. Sepuluh butir pernyataan SUS disusun secara bergantian antara pernyataan bernada positif (butir ganjil: 1, 3, 5, 7, 9) dan pernyataan bernada negatif (butir genap: 2, 4, 6, 8, 10). Prosedur perhitungan skor SUS adalah sebagai berikut:

1. Untuk setiap butir bernada positif (ganjil), kontribusi skor dihitung dengan mengurangi 1 dari respons pengguna: $(x_i - 1)$.
2. Untuk setiap butir bernada negatif (genap), kontribusi skor dihitung dengan mengurangi respons pengguna dari 5: $(5 - x_i)$.
3. Jumlahkan seluruh kontribusi skor dari 10 butir, lalu kalikan dengan 2,5 untuk mendapatkan skor akhir dalam rentang 0–100.

Secara matematis, rumus skor SUS dapat dituliskan sebagai:

$$\text{SUS} = 2{,}5 \times \left[ \sum_{i \in \text{ganjil}} (x_i - 1) + \sum_{j \in \text{genap}} (5 - x_j) \right] \tag{2.6}$$

Berdasarkan studi empiris yang dilakukan oleh Bangor, Kortum, and Miller (n.d.) terhadap lebih dari 2.000 survei, interpretasi skor SUS dikategorikan ke dalam tiga tingkatan penerimaan: skor di atas 70 dikategorikan sebagai *Acceptable* (dapat diterima), skor antara 50–70 dikategorikan sebagai *Marginal* (perlu perbaikan), dan skor di bawah 50 dikategorikan sebagai *Not Acceptable* (tidak dapat diterima). Selain itu, Bangor , dkk. juga memetakan skor SUS ke dalam skala penilaian deskriptif (*adjective rating*): *Excellent* ($\geq$ 85,5), *Good* (72,6–85,5), *OK* (50,9–72,5), *Poor* (25,1–50,9), dan *Worst Imaginable* ($\leq$ 25,0).

### 9. Pengujian Perangkat Lunak

Pengujian perangkat lunak (*software testing*) merupakan proses evaluasi terhadap suatu sistem untuk memastikan bahwa perilaku dan kinerjanya telah sesuai dengan spesifikasi yang diharapkan serta terbebas dari kesalahan (*bugs* atau *errors*). Pengujian ini bertujuan untuk meningkatkan keandalan, keamanan, dan kualitas perangkat lunak secara keseluruhan sebelum dirilis ke lingkungan produksi.

Secara garis besar, pengujian perangkat lunak dibagi menjadi beberapa metode dan tingkatan, antara lain: *Black-box testing* mengevaluasi fungsionalitas sistem tanpa memerlukan pengetahuan mengenai struktur kode atau logika internalnya. Pengujian dilakukan dengan memberikan *input* tertentu dan memverifikasi kesesuaiannya dengan *output* yang diharapkan.

*Unit testing* menguji komponen perangkat lunak terkecil, seperti fungsi, kelas, atau modul, secara terisolasi. Sementara itu, *integration testing* memverifikasi interaksi dan aliran data di antara modul yang saling terhubung. Pengujian otomatis (*automated testing*) menjalankan skenario yang telah diprogram menggunakan perangkat lunak khusus. Pendekatan ini mempercepat verifikasi dan membantu mencegah regresi (*regression*) ketika kode diperbarui. Dalam penelitian ini, pengujian otomatis menggunakan *Vitest*.

### 10. Unified Modeling Language (UML)

*Unified Modeling Language* (UML) merupakan bahasa pemodelan visual yang menyediakan notasi untuk menggambarkan dan mendokumentasikan struktur serta perilaku sistem perangkat lunak. UML membantu menyajikan kebutuhan dan rancangan sistem secara terstruktur sehingga dapat dipahami oleh pengembang maupun pemangku kepentingan. UML berfungsi sebagai bahasa pemodelan, bukan sebagai tahapan atau metodologi pengembangan perangkat lunak tertentu (Larman, 2005).

Diagram UML yang digunakan untuk memodelkan sistem dapat menggambarkan sudut pandang yang berbeda. Diagram *use case* menunjukkan fungsi sistem yang berinteraksi dengan aktor, diagram aktivitas (*activity*) menggambarkan alur kerja, diagram urutan (*sequence*) menunjukkan pertukaran pesan menurut urutan waktu, sedangkan diagram kelas (*class*) menunjukkan struktur kelas dan hubungan antarkelas. Simbol, nama, dan fungsi notasi pada diagram tersebut dirangkum pada Tabel 2.1.

**Tabel 2.1:** Simbol, Nama, dan Fungsi Notasi UML

| Simbol | Nama simbol | Fungsi simbol |
|---|---|---|
|  | Aktor | Mewakili peran pengguna atau sistem eksternal yang berinteraksi dengan sistem; aktor berada di luar batas sistem dan dapat memulai atau berpartisipasi dalam suatu fungsi. |
|  | *Use case* | Menyatakan layanan atau tujuan yang disediakan sistem bagi aktor. Nama di dalam elips biasanya ditulis sebagai frasa kerja, misalnya "Mengunggah audio". |
|  | Batas sistem | Persegi panjang yang membatasi ruang lingkup sistem; nama sistem dicantumkan pada bagian atas dan *use case* diletakkan di dalamnya, sedangkan aktor berada di luarnya. |
|  | Asosiasi | Garis penuh yang menghubungkan aktor dengan *use case* untuk menunjukkan partisipasi atau komunikasi. Pada diagram kelas, asosiasi yang sama menghubungkan kelas-kelas yang memiliki hubungan struktural. |
|  | Relasi *include* | Garis putus-putus berpanah dengan stereotipe «include» dari *use case* asal menuju *use case* yang disertakan. Menunjukkan bahwa perilaku *use case* tujuan selalu dipanggil sebagai bagian dari *use case* asal. |
| Berlanjut ke halaman berikutnya | | |

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| Simbol | Nama simbol | Fungsi simbol |
|---|---|---|
|  | Relasi *extend* | Garis putus-putus berpanah dengan stereotipe «extend» dari *use case* tambahan menuju *use case* dasar. Menunjukkan perilaku tambahan yang dijalankan hanya jika kondisi atau titik ekstensi tertentu terpenuhi. |
|  | Simpul awal (*initial node*) | Lingkaran hitam penuh yang menandai titik awal aliran aktivitas; umumnya hanya ada satu simpul awal untuk satu alur proses. |
|  | Aksi (*action*) | Persegi panjang bersudut tumpul yang menyatakan satu langkah kerja atau aktivitas dalam proses; uraian aksi ditulis di dalam simbol. |
|  | Simpul keputusan atau penggabungan | Bentuk wajik yang digunakan untuk percabangan berdasarkan kondisi (dengan beberapa aliran keluar) atau menggabungkan aliran alternatif (dengan beberapa aliran masuk). Kondisi dapat ditulis sebagai *guard* pada aliran keluar. |
|  | Aliran kontrol (*control flow*) | Garis penuh berpanah yang menunjukkan urutan perpindahan kendali dari simpul aktivitas sebelumnya ke aksi atau simpul berikutnya. |
| Berlanjut ke halaman berikutnya | | |

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| Simbol | Nama simbol | Fungsi simbol |
|---|---|---|
|  | Simpul *fork*/*join* | Batang tebal untuk memecah satu aliran menjadi beberapa aktivitas paralel (*fork*) atau menyinkronkan beberapa aliran paralel sebelum proses berlanjut (*join*). |
|  | Simpul akhir aktivitas | Lingkaran berbingkai dengan titik hitam di tengah yang menandai berakhirnya seluruh aktivitas pada alur tersebut. Berbeda dari simpul akhir aliran, simpul ini menghentikan aktivitas secara keseluruhan. |
|  | Partisipan (*participant*) | Persegi panjang di bagian atas diagram urutan yang berisi nama aktor atau objek yang terlibat dalam interaksi; garis hidup partisipan dimulai dari simbol ini. |
|  | Garis hidup (*lifeline*) | Garis vertikal putus-putus di bawah partisipan yang menunjukkan keberadaan partisipan sepanjang waktu interaksi; waktu dibaca dari atas ke bawah. |
|  | Aktivasi (*activation*) | Persegi panjang tipis pada garis hidup yang menunjukkan rentang waktu ketika partisipan sedang menjalankan operasi atau memiliki kendali. |
|  | Pesan sinkron | Garis penuh berujung panah penuh yang menunjukkan pemanggilan operasi; pengirim menunggu hingga penerima menyelesaikan operasi dan mengembalikan kendali. |

Berlanjut ke halaman berikutnya

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| Simbol | Nama simbol | Fungsi simbol |
|---|---|---|
|  | Pesan balasan (*return message*) | Garis putus-putus berpanah yang menunjukkan kembalinya kendali atau hasil operasi dari penerima kepada pengirim pesan. |
|  | Kelas (*class*) | Persegi panjang yang dibagi menjadi nama kelas, atribut, dan operasi. Bagian atribut dan operasi dapat dikosongkan jika tidak diperlukan dalam model. |
|  | Asosiasi antarkelas | Garis penuh yang menunjukkan hubungan struktural antarkelas. Nama relasi, arah navigasi, dan multiplisitas pada ujung garis dapat ditambahkan untuk merinci hubungan. |
|  | Agregasi | Garis asosiasi dengan wajik kosong di sisi keseluruhan; menyatakan hubungan keseluruhan-bagian yang lemah, sehingga bagian tetap dapat berdiri sendiri ketika keseluruhan dihapus. |
|  | Komposisi | Garis asosiasi dengan wajik penuh di sisi pemilik; menyatakan hubungan keseluruhan-bagian yang kuat, dengan siklus hidup bagian bergantung pada keseluruhan. |
|  | Generalisasi | Garis penuh dengan segitiga kosong yang menunjuk ke kelas induk; menunjukkan pewarisan, sehingga kelas khusus mewarisi karakteristik kelas umum. |
| Berlanjut ke halaman berikutnya | | |

**Tabel 2.1 – Lanjutan dari halaman sebelumnya**

| Simbol | Nama simbol | Fungsi simbol |
|---|---|---|
|  | Dependensi | Garis putus-putus berpanah terbuka dari kelas yang bergantung menuju kelas yang menjadi pemasok; perubahan pada pemasok dapat memengaruhi kelas yang bergantung. |

*Sumber:* Diadaptasi dari Larman (2005).

## B. Literatur Review

Tinjauan pustaka dilakukan dengan menganalisis berbagai penelitian terdahulu yang memiliki relevansi dengan pemrosesan bahasa alami dan arsitektur sistem terdistribusi. Data literatur tersebut dirangkum dalam Tabel 2.2 untuk memberikan gambaran komprehensif mengenai metode, tujuan, dan temuan utama dari setiap penelitian yang dirujuk.

**Tabel 2.2:** Ringkasan Literatur Review

| Judul | Penulis | Metode Penelitian | Tujuan Penelitian | Hasil Penelitian |
|---|---|---|---|---|
| User-centered Design Methods in Data-Intensive Software... | Riebel dkk. (2023) | *Literature Review* | Menganalisis integrasi UCD dalam AI | UCD meningkatkan empati dan definisi masalah dalam pengembangan AI. |
| Benchmarking open source and paid services for speech to text... | Ferraro dkk. (2023) | *Benchmark & WER Analysis* | Membandingkan performa layanan ASR berbayar dan open-source. | Layanan berbayar lebih akurat, namun performa sangat bergantung pada variasi input teks. |
| Berlanjut ke halaman berikutnya | | | | |

**Tabel 2.2 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Evaluating Open-Source ASR Systems... | Imai dkk. (2025) | Evaluasi Eksperimental | Mengukur akurasi ASR pada audio bising | Perlu *robust error correction* untuk mengatasi bias demografis. |
| End-to-End Speech Recognition: A Survey | Prabhavalkar dkk. (2023) | *Survey* Literatur | Memetakan perkembangan E2E ASR | Model E2E mereduksi WER hingga 50% dibanding HMM tradisional. |
| Transformers in Speech Processing: A Survey | Latif dkk. (2025) | *Survey* Literatur | Menganalisis *Transformer* dalam audio | *Transformer* unggul dalam memodelkan dependensi jangka panjang. |
| Multilingual DistilWhisper... | Ferraz dkk. (2024) | Eksperimental (*Distillation*) | Meringankan model Whisper untuk bahasa langka | *DistilWhisper* meningkatkan performa bahasa *under-represented* dengan *overhead* minim. |
| Does ChatGPT and Whisper Make Humanoid Robots More Relatable? | Chen dkk. (2024) | Evaluasi Pengguna | Membandingkan Whisper vs Google ASR pada robot | Whisper memiliki WER terendah (1.71%) dan paling disukai oleh pengguna. |

Berlanjut ke halaman berikutnya

**Tabel 2.2 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Quantization for OpenAI's Whisper Models... | Andreyev (2025) | Analisis Komparatif | Mengukur dampak kuantisasi pada latensi | Kuantisasi INT8 mereduksi size 45% dan latensi 19% tanpa degradasi akurasi. |
| A Self-Training Approach for Whisper to Enhance Long Dysarthric Speech... | Wang dkk. (2025) | *Self-training method* | Meningkatkan ASR untuk penderita disartria | Metode *self-training* meningkatkan performa pada ucapan penderita disabilitas. |
| Accurate Speech Recognition AI using Model of Deep Learning... | Isyanto dkk. (2024) | Eksperimental CNN | Membangun ASR untuk keamanan perbankan | Algoritma CNN mencapai akurasi 99% untuk pengenalan kata kunci keamanan. |
| Evaluating OpenAI's Whisper ASR... | Graham and Roll (2024) | Analisis Performa | Mengevaluasi Whisper pada berbagai aksen | Whisper lebih akurat pada aksen asli (*native*) dibandingkan aksen *non-native*. |
| The Role of TypeScript in Enhancing Development... | Sekhar Emmann (2021) | *Empirical Analysis* | Menganalisis peran TypeScript dalam pengembangan aplikasi *web* modern. | TypeScript secara signifikan mengurangi kesalahan *runtime* dan meningkatkan skalabilitas. |

Berlanjut ke halaman berikutnya

**Tabel 2.2 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| A Comparative Study of Agile and Waterfall Software Development... | John dkk. (2024) | Analisis Komparatif | Membandingkan *Agile* vs *Waterfall* | *Agile* lebih unggul dalam kepuasan pelanggan dan kecepatan pengembangan. |
| Turning Whisper into Real-Time Transcription System | Macháček , dkk. (2023) | Implementasi | Mengembangkan sistem transkripsi *real-time* berbasis Whisper. | *Whisper-Streaming* mencapai latensi 3,3 detik dengan akurasi yang tetap terjaga. |
| WhisperX: Time-Accurate Speech Transcription... | Bain , dkk. (2023) | Pengembangan Sistem | Penyelarasan waktu tingkat kata pada Whisper | VAD dan *forced alignment* memungkinkan transkripsi 12x lebih cepat. |
| Bridging the Gap Between Generalized ASR and Domain-Specific... | Nelson dkk. (2025) | Integrasi Vosk | Menyesuaikan ASR untuk domain spesifik | Language modeling pada Vosk efektif untuk istilah medis dan hukum. |
| A semi-automatic pipeline for transcribing child speech | Christodoulidou , dkk. (2025) | Evaluasi *Pipeline* | Membangun *pipeline* transkripsi suara anak | Koreksi manual sebelum *forced alignment* meningkatkan kualitas data akustik. |

Berlanjut ke halaman berikutnya

**Tabel 2.2 – Lanjutan dari halaman sebelumnya**

| **Judul** | **Penulis** | **Metode Penelitian** | **Tujuan Penelitian** | **Hasil Penelitian** |
|---|---|---|---|---|
| Comparing ASR Systems in Context of Speech Disfluencies | Teleki dkk. (2024) | Analisis Korpus Besar | Mengevaluasi Google ASR vs Whisper | Whisper lebih unggul dalam menangani interjeksi dan keraguan bicara. |
| Perancangan Antarmuka Pengguna Aplikasi Informasi Siswa... | Pratiwi dkk. (2022) | *User Centered Design* (UCD) | Merancang aplikasi informasi sekolah | Metode UCD menghasilkan desain yang 100% dapat digunakan oleh target *user*. |
| Estonian Speech Recognition and Transcription Editing Service | Olev and Alumäe (2022) | Pengembangan Layanan | Membangun sistem transkripsi berbasis wav2vec2.0 | Sistem berbasis E2E mencapai WER 6.9% dan mendukung *editing* kolaboratif. |

Berdasarkan tinjauan literatur di atas, penelitian ini memadukan tiga pilar utama yang saling terkait. Pilar pertama adalah mesin inferensi transkripsi berupa model OpenAI *Whisper* yang dioptimasi menggunakan pustaka *Faster-Whisper* (CTranslate2) dan *pipeline WhisperX* untuk menangani berkas audio berdurasi panjang secara efisien. Pilar kedua adalah fondasi arsitektur terdistribusi berbasis *Next.js* dan *Python Worker* yang dihubungkan melalui antrean tugas asinkron *Redis BullMQ* untuk memastikan stabilitas operasional mandiri (*self-hosted*) pada infrastruktur lokal. Pilar ketiga adalah perancangan antarmuka penyuntingan terintegrasi dengan pendekatan *User-Centered Design* (UCD) yang dievaluasi menggunakan instrumen *System Usability Scale* (SUS). Ketiga pilar ini menjadi

kerangka acuan utama perancangan sistem Skriptor yang dibahas pada Bab III.

# BAB III
# METODE PENELITIAN

## A. Jenis Penelitian

Penelitian ini termasuk dalam kategori penelitian rekayasa perangkat lunak dengan pendekatan *Research and Development* (R&D). Pengembangan sistem mengikuti model *waterfall* yang berjalan secara sekuensial melalui tahap perancangan, implementasi, dan pengujian, dengan penekanan iteratif khusus pada fase perancangan antarmuka. Antarmuka koreksi pada sistem dirancang menggunakan pendekatan *User-Centered Design* (UCD), yaitu dimulai dari pemahaman kebutuhan pengguna, perancangan *wireframe*, hingga validasi desain sebelum implementasi. Pengujian sistem mencakup tiga aspek. Pertama, pengujian fungsionalitas sistem melalui *black-box testing* manual serta *unit testing* otomatis menggunakan *Vitest* untuk memastikan kode berjalan andal. Kedua, pengujian teknis berupa *benchmarking* yang mengukur kecepatan inferensi, konsumsi VRAM, dan akurasi transkripsi (WER dan CER) dari berbagai varian model OpenAI *Whisper*. Ketiga, pengujian non-teknis berupa evaluasi kegunaan (*usability*) menggunakan kuesioner *System Usability Scale* (SUS).

## B. Setting Penelitian

Penelitian ini berlangsung dengan memanfaatkan infrastruktur *cloud* dan perangkat keras lokal. Tugas inferensi model AI yang intensif dijalankan pada *remote pod* dari layanan *Runpod* yang menyediakan GPU NVIDIA RTX A5000 dengan VRAM 24GB. Sementara itu, aplikasi web beserta layanan pendukungnya berjalan pada server lokal. Pemilihan infrastruktur ini didasari kebutuhan akan lingkungan komputasi yang terkontrol dan fleksibel, sehingga spesifikasi perangkat keras dapat disesuaikan dengan kebutuhan model kecerdasan buatan skala besar. Penelitian direncanakan berlangsung selama delapan bulan, dimulai pada bulan Maret hingga Oktober 2026.

## C. Prosedur Penelitian

Prosedur penelitian disusun secara sistematis agar setiap tahap menghasilkan luaran yang dapat divalidasi pada tahap berikutnya. Rincian setiap tahapan dijelaskan sebagai berikut: Tahap studi literatur diawali dengan meninjau dokumentasi model *Whisper*, arsitektur sistem terdistribusi berbasis antrean (*queue-based*), serta metrik evaluasi ASR. Teori *human-computer interaction* juga dikaji untuk mendukung perancangan antarmuka koreksi yang ergonomis. Referensi ilmiah yang relevan dikumpulkan sebagai dasar penelitian.

Pada tahap perancangan sistem, disusun skema interaksi antara *Next.js* sebagai orkestrator tugas dan *Python Worker* sebagai unit eksekusi GPU. Perancangan mencakup Redis *Pub/Sub* untuk pelaporan status secara *real-time*, struktur data laporan *benchmarking*, serta diagram basis data. Antarmuka koreksi dirancang menggunakan pendekatan *User-Centered Design* (UCD), dimulai dari identifikasi kebutuhan calon pengguna hingga penyusunan dan validasi *wireframe* serta *prototype*.

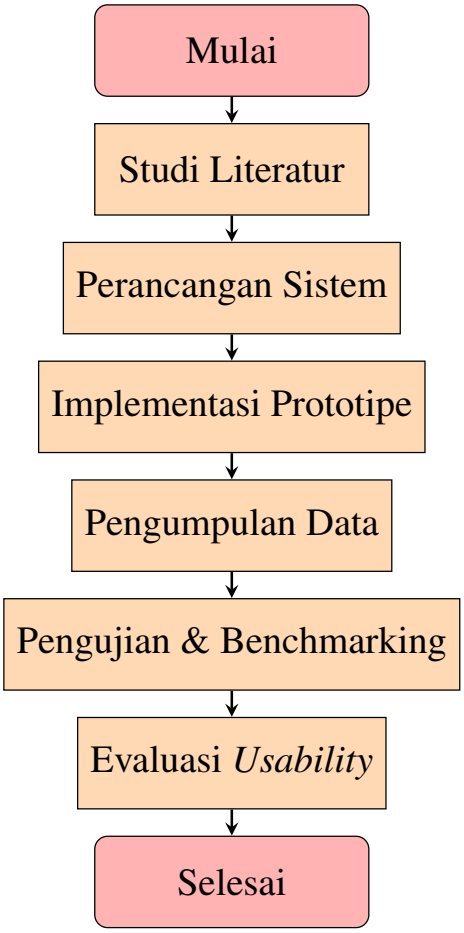
Implementasi prototipe dilakukan dengan mengintegrasikan komponen yang telah dirancang. *Worker* dikonfigurasi menggunakan *pipeline WhisperX* yang menggabungkan *Faster-Whisper*, VAD, dan *forced alignment*. Teknik kuantisasi diterapkan untuk mengoptimalkan VRAM pada RTX A5000. Implementasi juga mencakup antarmuka web untuk mengunggah audio dan memantau progres.

Pada tahap pengumpulan data, sebanyak 58 berkas audio utuh per episode disiapkan dari korpus *Mozilla Data Collective* (MDC), yang terdiri atas 16 episode *Homostoria* dan 42 episode *Hari Minggoean*. Berkas acuan *ground truth* berformat *TSV* memuat 11.540 segmen transkrip beserta anotasi waktu. Kesesuaian durasi audio dan teks manual divalidasi sebelum evaluasi.

Pengujian sistem dilakukan melalui *black-box testing* manual dan pengujian unit menggunakan *Vitest*. Skenario pengujian mencakup pengunggahan audio, penolakan format yang tidak didukung, pemantauan progres, dan penyimpanan hasil koreksi. Setelah sistem stabil, transkripsi diuji pada varian *tiny, base, small,*

*medium, large-v2, large-v3,* dan *large-v3-turbo*. Waktu inferensi, RTF, penggunaan memori, dan hasil transkripsi mentah dicatat dalam lingkungan terkontrol.

Evaluasi *usability* dilakukan menggunakan kuesioner *System Usability Scale* (SUS) terhadap 30 responden yang mewakili mahasiswa, akademisi, dan pembuat konten digital. Responden dipilih secara *purposive sampling*, kemudian diminta menjalankan tugas mengunggah audio, memantau progres, dan mengoreksi transkripsi pada antarmuka Skriptor. Setelah menyelesaikan tugas, responden mengisi sepuluh pernyataan SUS berskala Likert 5 poin. Skor yang diperoleh dihitung dan diinterpretasikan berdasarkan kategori standar Bangor , dkk. (n.d.); Brooke (n.d.).




**Gambar 3.1:** Alur Penelitian (*Flowchart*)

## D. Data dan Sumber Data

Bahan penelitian utama berupa data audio dan transkrip referensi (*ground truth*). Data penelitian menggunakan korpus *Mozilla Data Collective* (MDC) yang terdiri atas 16 episode *Homostoria* dan 42 episode *Hari Minggoean* dalam bahasa Indonesia. Dengan demikian, data berjumlah 58 berkas audio utuh per episode. Rekaman tersebut merupakan percakapan alami dengan total durasi 23,5 jam dan memiliki variasi intonasi, logat lokal, serta gangguan latar belakang (*noise*).

Berkas *TSV* (*tab-separated values*) digunakan sebagai *ground truth*. Berkas ini memuat 11.540 segmen transkrip manual beserta anotasi waktu dan berfungsi sebagai acuan untuk menghitung kesalahan pada tingkat kata maupun karakter. Rincian statistik dataset disajikan pada Tabel 3.1.

**Tabel 3.1:** Statistik Dataset Penelitian

| **Nama Siniar** | **Jumlah Berkas** | **Jumlah Segmen** | **Total Durasi (Menit)** |
|---|---|---|---|
| Homostoria | 16 | 5.163 | 678,02 |
| Hari Minggoean | 42 | 6.377 | 735,43 |
| **Total** | **58** | **11.540** | **1.413,45** |

## E. Alat dan Instrumen Penelitian

Instrumen penelitian terdiri dari perangkat keras dan perangkat lunak yang dipilih untuk mendukung arsitektur sistem terdistribusi berperforma tinggi. Pemilihan komponen ini didasarkan pada kebutuhan skalabilitas (*scalability*) dan efisiensi dalam menangani model AI yang berat. Perangkat keras penelitian meliputi NVIDIA RTX A5000 dengan VRAM 24 GB melalui *Runpod* untuk inferensi AI, *Thin Client HP t630* dengan RAM 8 GB sebagai server aplikasi lokal, serta laptop *Lenovo IdeaPad Slim 3 14ALC6* dengan prosesor *AMD Ryzen 5 5500U*, RAM 20 GB, dan penyimpanan *NVMe SSD* untuk pengembangan serta pengujian lokal.

Perangkat lunak yang digunakan meliputi *Visual Studio Code*, *Git*, dan *Bun Runtime*. Lapisan aplikasi dibangun dengan *TypeScript*, *Next.js* 16, *React* 19, *Tailwind CSS* v4, *shadcn/ui*, dan *Drizzle ORM*. Infrastruktur data menggunakan *PostgreSQL*, *Redis* dengan *BullMQ*, serta *Garage S3*. Pemrosesan audio menggunakan *PyTorch*, *WhisperX*, *Faster-Whisper*, *CTranslate2*, *pyannote.audio*

3.1, dan *jiwer*, sedangkan pengujian unit dilakukan dengan *Vitest*.

Instrumen evaluasi yang digunakan adalah kuesioner *System Usability Scale* (SUS), yang terdiri atas sepuluh pernyataan berskala Likert 5 poin Brooke (n.d.). Kuesioner dibagikan setelah responden menyelesaikan skenario tugas pada antarmuka Skriptor. Responden berjumlah 30 orang dan dipilih secara *purposive sampling* dari kelompok mahasiswa, akademisi, serta pembuat konten digital yang memiliki kebutuhan transkripsi.

## F. Teknik Analisis Data

Analisis data dalam penelitian ini dilakukan melalui lima pendekatan yang saling melengkapi guna mengevaluasi aspek akurasi, efisiensi, kegunaan, dan fungsionalitas sistem secara komprehensif.

### 1. Pra-pemrosesan dan Normalisasi Teks

Sebelum dilakukan perbandingan antara teks hipotesis (keluaran ASR) dan teks referensi (*ground truth*), kedua teks terlebih dahulu melewati tahap normalisasi. Proses normalisasi meliputi pengubahan seluruh karakter ke huruf kecil (*case-folding*), penghapusan tanda baca menggunakan ekspresi reguler, dan penyeragaman spasi. Secara formal, fungsi normalisasi didefinisikan sebagai berikut:

$$f(\text{teks}) = \text{lowercase}(\text{strip}(\text{remove_punctuation}(\text{teks}))) \tag{3.1}$$

Teks yang telah dinormalisasi kemudian digunakan sebagai masukan untuk penghitungan metrik akurasi.

### 2. Analisis Akurasi ASR (WER dan CER)

Akurasi transkripsi diukur menggunakan dua metrik utama yang berbasis jarak *Levenshtein*. *Word Error Rate* (WER) mengukur tingkat kesalahan pada level kata dengan persamaan:

$$\text{WER} = \frac{S+D+I}{N} = \frac{S+D+I}{S+D+C} \tag{3.2}$$

dengan $S$ = jumlah substitusi, $D$ = jumlah penghapusan (*deletions*), $I$ = jumlah penyisipan (*insertions*), $C$ = jumlah kata benar (*correct*), dan $N$ = total kata pada teks referensi. *Character Error Rate* (CER) mengukur tingkat kesalahan pada level karakter menggunakan jarak *Levenshtein*:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c} \tag{3.3}$$

dengan $S_c$, $D_c$, dan $I_c$ berturut-turut adalah jumlah substitusi, penghapusan, dan penyisipan karakter, sedangkan $N_c$ adalah jumlah karakter pada teks referensi. Perhitungan WER dan CER dilakukan menggunakan pustaka *jiwer* pada *Python*. Nilai yang lebih rendah menunjukkan akurasi transkripsi yang lebih baik.

### 3. Analisis Efisiensi Komputasi

Efisiensi komputasi diukur melalui *Real-Time Factor* (RTF) dan penggunaan VRAM puncak. RTF merupakan rasio antara waktu inferensi dan durasi audio yang diproses:

$$\text{RTF} = \frac{\Delta t_{\text{inferensi}}}{\Delta t_{\text{audio}}} \tag{3.4}$$

Nilai RTF $< 1$ menunjukkan bahwa sistem memproses audio lebih cepat dari waktu nyata. Kecepatan pemrosesan juga dapat dinyatakan sebagai *Speed Factor* (Speed $= 1/\text{RTF}$). Penggunaan VRAM puncak (*Peak VRAM*) dicatat melalui *polling* perintah `nvidia-smi` atau fungsi `torch.cuda.memory_reserved()` pada *PyTorch*. Data tersebut digunakan untuk menentukan kelayakan model pada berbagai spesifikasi GPU.

### 4. Analisis Kegunaan (*Usability*) dengan SUS

Evaluasi kegunaan antarmuka dilakukan menggunakan kuesioner *System Usability Scale* (SUS) yang terdiri dari 10 butir pernyataan berskala Likert 1–5 Brooke (n.d.). Kuesioner didistribusikan kepada 30 responden. Skor SUS untuk

setiap responden dihitung dengan formula:

$$\text{SUS} = 2{,}5 \times \left[ \sum_{i \in \text{ganjil}} (x_i - 1) + \sum_{j \in \text{genap}} (5 - x_j) \right] \tag{3.5}$$

dengan $x_i$ adalah skor butir ganjil (pernyataan positif) dan $x_j$ adalah skor butir genap (pernyataan negatif). Rentang skor SUS adalah 0–100. Rata-rata dan standar deviasi skor SUS kemudian dihitung dan diinterpretasikan berdasarkan skala penerimaan (*Acceptability Ranges*), skala huruf (*Grade Scale*), dan peringkat kata sifat (*Adjective Rating*) dari Bangor , dkk. (n.d.).

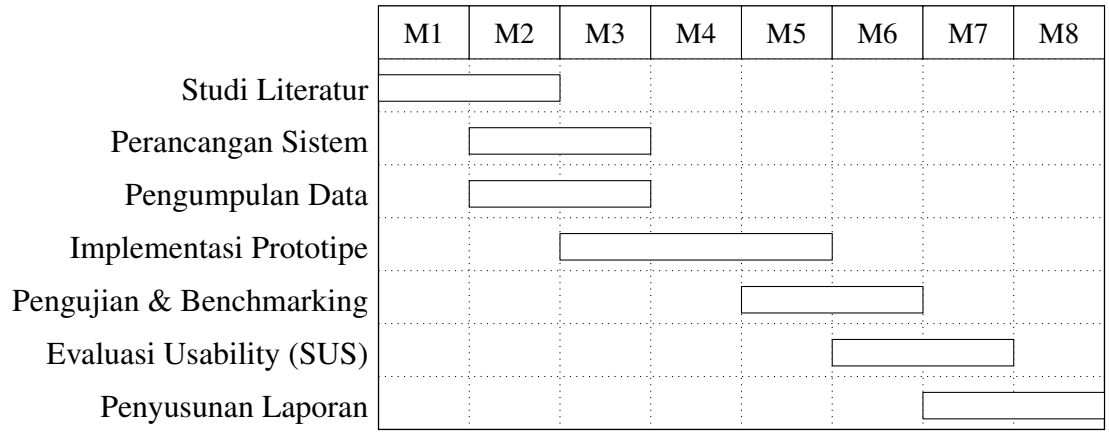
### 5. Analisis Pengujian Fungsional

Pengujian fungsional dilakukan melalui pendekatan kotak hitam manual (*manual black-box testing*). Setiap fitur utama sistem diuji berdasarkan matriks skenario pengujian yang memuat: identifikasi skenario, kondisi masukan (*input*), langkah pengujian, respons sistem yang diharapkan (*expected result*), dan hasil pengamatan aktual (*actual result*). Setiap skenario dinilai dengan status "Berhasil" atau "Gagal" berdasarkan kesesuaian antara keluaran aktual dan keluaran yang diharapkan.

## G. Jadwal Penelitian

Penelitian direncanakan berlangsung selama delapan bulan. Rincian jadwal penelitian disajikan dalam bentuk *Gantt Chart* pada Tabel 3.2.

**Tabel 3.2:** Jadwal Penelitian (*Gantt Chart*)



# BAB IV
# HASIL DAN PEMBAHASAN

## A. Hasil Penelitian

Hasil penelitian akan disajikan di sini dalam bentuk tabel, gambar, atau grafik sesuai dengan metrik yang telah ditetapkan.

## B. Pembahasan

Interpretasi hasil dan perbandingan dengan studi terdahulu akan dijelaskan pada bagian ini.

# BAB V
# PENUTUP

## A. Kesimpulan

Kesimpulan menjawab pertanyaan penelitian secara ringkas dan padat.

## B. Saran

Rekomendasi untuk penelitian selanjutnya akan dicantumkan di sini.

# DAFTAR PUSTAKA


Andreyev, A. (n.d.). *Quantization for OpenAI's whisper models: A comparative analysis.* arXiv. Diakses pada 2026-04-14http://arxiv.org/abs/2503.09905 doi: 10.48550/arXiv.2503.09905

Bain, M., Huh, J., Han, T., & Zisserman, A. (n.d.). WhisperX: Time-accurate speech transcription of long-form audio. In *INTERSPEECH 2023* (pp. 4489–4493). ISCA. Diakses pada 2026-04-14https://www.isca-archive.org/interspeech_2023/bain23_interspeech.html doi: 10.21437/Interspeech.2023-78

Bangor, A., Kortum, P. T., & Miller, J. T. (n.d.). An empirical evaluation of the system usability scale. , *24*(6), 574–594. Diakses pada 2026-05-23https://www.tandfonline.com/doi/full/10.1080/10447310802205776 doi: 10.1080/10447310802205776

Brooke, J. (n.d.). SUS - a quick and dirty usability scale.

Christodoulidou, P., Tanner, J., Stuart-Smith, J., McAuliffe, M., Murali, M., Smith, A., ... Kuschmann, A. (n.d.). A semi-automatic pipeline for transcribing and segmenting child speech. In *Interspeech 2025* (pp. 4278–4282). ISCA. Diakses pada 2026-04-14https://www.isca-archive.org/interspeech_2025/christodoulidou25_interspeech.html doi: 10.21437/Interspeech.2025-1030

Fathony, I. A. N., Mareta, A., Adiana, B. E., Wardhani, O., & Halim, A. (n.d.). Optimisasi whisper speech-to-text bahasa indonesia dengan hybrid cloud dan multi-engine. , *18*(1).

Ferraro, A., Galli, A., La Gatta, V., & Postiglione, M. (n.d.). Benchmarking open source and paid services for speech to text: an analysis of quality and input variety. , *6*, 1210559. Diakses pada 2026-04-14https://www.frontiersin.org/articles/10.3389/fdata.2023.1210559/full doi: 10.3389/fdata.2023.1210559

Gaur, Y., Lasecki, W. S., Metze, F., & Bigham, J. P. (n.d.). The effects of automatic

speech recognition quality on human transcription latency. In *Proceedings of the 13th international web for all conference* (pp. 1–8). ACM. Diakses pada 2026-04-14https://dl.acm.org/doi/10.1145/2899475.2899478 doi: 10.1145/2899475.2899478

Graham, C., & Roll, N. (n.d.). Evaluating OpenAI's whisper ASR: Performance analysis across diverse accents and speaker traits. , *4*(2), 025206. Diakses pada 2026-04-14https://pubs.aip.org/jel/article/4/2/025206/3267247/Evaluating-OpenAI-s-Whisper-ASR-Performance doi: 10.1121/10.0024876

Imai, S., Chowdhury, T., & Stent, A. J. (n.d.). Evaluating open-source ASR systems: Performance across diverse audio conditions and error correction methods.

Larman, C. (2005). *Applying UML and patterns: an introduction to object-oriented analysis and design and iterative development* (3rd ed ed.). Prentice Hall PTR, c2005.

Macháček, D., Dabre, R., & Bojar, O. (n.d.). Turning whisper into real-time transcription system. In *Proceedings of the 13th international joint conference on natural language processing and the 3rd conference of the asia-pacific chapter of the association for computational linguistics: System demonstrations* (pp. 17–24). Association for Computational Linguistics. Diakses pada 2026-04-14https://aclanthology.org/2023.ijcnlp-demo.3 doi: 10.18653/v1/2023.ijcnlp-demo.3

Olev, A., & Alumäe, T. (n.d.). Estonian speech recognition and transcription editing service. , *10*(3). Diakses pada 2026-04-14http://www.bjmc.lu.lv/fileadmin/user_upload/lu_portal/projekti/bjmc/Contents/10_3_14_Olev.pdf doi: 10.22364/bjmc.2022.10.3.14

Prabhavalkar, R., Hori, T., Sainath, T. N., Schlüter, R., & Watanabe, S. (n.d.). *End-to-end speech recognition: A survey.* arXiv. Diakses pada 2026-04-14http://arxiv.org/abs/2303.03329 doi: 10.48550/arXiv.2303.03329

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (n.d.).

Robust speech recognition via large-scale weak supervision.

Riebel, J. A. (n.d.). User-centered design methods in data-intensive software development processes: A state-of-the-art review.

Sekhar Emmanni, P. (n.d.). The role of TypeScript in enhancing development with modern JavaScript frameworks. , *10*(2), 1738–1741. Diakses pada 2026-04-14https://www.ijsr.net/getabstract.php?paperid=SR24401234212 doi: 10.21275/SR24401234212

Teleki, M., Dong, X., Kim, S., & Caverlee, J. (n.d.). Comparing ASR systems in the context of speech disfluencies. In *Interspeech 2024* (pp. 4548–4552). ISCA. Diakses pada 2026-04-14https://www.isca-archive.org/interspeech_2024/teleki24_interspeech.html doi: 10.21437/Interspeech.2024-1270